Retrieval, measured

A first-principles walkthrough of the retrieval-ablation project

---

Part I - The problem, from nothing

1. What this project actually is

Imagine you have 120 annual reports from large American companies. Together they run to about 27,500 pages. Someone asks:

> What was Apple's research and development expense in fiscal 2025?

The answer is one number, sitting in one row of one table, in one of those 120 documents. A system that answers this question has to do two separate jobs:

1. **Find** the handful of paragraphs, out of tens of thousands, that contain the answer.
2. **Write** a reply based on what it found.

Almost every published project that does this measures only the second job. It shows you an answer and asks whether the answer looks right. That is a mistake, and understanding why is the whole point of this project.

If step 1 handed step 2 the wrong pages, then step 2 is being asked to write an answer from material that does not contain one. It will either refuse, or invent something. No amount of improving step 2 fixes that. The two jobs fail for completely different reasons and are repaired by completely different means, so measuring them together tells you almost nothing about what to do next.

This project measures step 1 on its own. That is the entire idea. Everything else follows from it.

2. Why "measuring step 1 on its own" is harder than it sounds

To score the finding step by itself, you need to already know the right answer - not the answer *text*, but which specific passage contains it. You need a list like:

question	the passage that answers it
"Apple R&D expense 2025?"	document aapl-10-k-2025, characters 1,215,596-1,216,693

That list is called a **labelled evaluation set**, and building one is most of the work. With it, you can run any finding-system over the questions, see where it ranked the correct passage, and score it - **with no language model involved at all**. That makes the measurement free, instant, and perfectly repeatable.

Without it, you are guessing.

3. How text gets found: the two families

There are two fundamentally different ways to find text.

Lexical search - matching words

The classic method. You build an index of which words appear in which passages, then for a query you look up passages containing the query's words and rank them.

The standard scoring function is **BM25**. Three ideas in it:

1. **Rare words matter more.** A passage containing "amortisation" tells you more than one containing "the". This is *inverse document frequency* - a word in few documents gets a high weight.
2. **Repetition helps, with diminishing returns.** A passage using "amortisation" five times is more likely to be about it than one using it once - but twenty times is not four times better than five. This is *term-frequency saturation*.
3. **Long passages need discounting.** A very long passage contains more words by accident, so it would win by length alone without a correction. This is *length normalisation*.

BM25 is old, simple, fast, and remains a genuinely strong baseline. It has one fundamental limitation: it can only match words that are actually there. Ask about "R&D spending" when the filing says "research and development expense" and BM25 sees almost nothing in common.

Dense search - matching meaning

The modern method. A neural network reads a passage and emits a list of numbers - say 1,024 of them - called an **embedding** or **vector**. The network is trained so that passages meaning similar things get similar lists. You embed every passage once, then embed the query, then find the passages whose vectors point in most nearly the same direction.

"Direction" is literal. Each vector is an arrow in 1,024-dimensional space. Similarity is measured by the angle between two arrows - the **cosine similarity**. Identical direction scores 1.0, perpendicular scores 0.0.

Dense search handles "R&D spending" versus "research and development expense" easily, because the network learned they mean the same thing. Its weakness is the mirror image of BM25's: it is fuzzy. Ask for a specific number, or a specific year, and a dense retriever will happily return something that is *about* the right topic but concerns the wrong fiscal year.

Hybrid - using both

Since the two fail differently, combining them should help. The difficulty is that their scores are not comparable: BM25 produces unbounded sums of word weights, cosine similarity lives between -1 and 1. Adding them is meaningless.

The standard fix is **Reciprocal Rank Fusion (RRF)**. Throw away the scores entirely and keep only the *positions*. A passage ranked 1st by one system and 3rd by the other gets `1/(k+1) + 1/(k+3)`, with `k` a constant (60 by convention). Sum across systems, sort. Because only ranks are used, the incomparable-scores problem simply disappears.

The cost of discarding scores is real: RRF cannot tell a confident first place from a marginal one.

Reranking - the expensive second look

All the above are **bi-encoders**: query and passage are processed separately and compared at the end. That is fast - passages are embedded once, in advance - but the model never sees the query and passage *together*, so it cannot reason about how they relate.

A **cross-encoder** reads the query and one passage jointly and outputs a relevance score directly. Much more accurate, and far more expensive: one full neural network run per passage, with nothing precomputable.

So cross-encoders are used as a **second stage**. A cheap retriever produces a shortlist of, say, 50 candidates; the cross-encoder rescores just those and reorders them. This is widely described as the highest-value component in production retrieval. Whether that is true *on a given corpus* is exactly the sort of claim this project exists to check rather than repeat.

4. How finding is scored

Three measurements, each answering a different question.

Recall@50 - "did it find the answer at all?"

Of the passages that are genuinely relevant, what fraction appear in the top 50? Simple, and it measures a **ceiling**: anything not in the top 50 cannot be used by a later stage, no matter how clever that stage is.

MRR - "how far down was the first good hit?"

Mean Reciprocal Rank. If the first relevant passage is at position 1, score 1.0. Position 2 scores 0.5, position 4 scores 0.25. Averaged over queries. It cares only about the first correct hit.

nDCG@10 - "is the top of the list good?"

Normalised Discounted Cumulative Gain, the standard in information retrieval. Built in three steps:

1. **Gain.** Each passage has a relevance grade. A grade of 2 contributes `2^2 - 1 = 3`; a grade of 1 contributes `2¹ - 1 = 1`; irrelevant contributes 0.
2. **Discount.** A hit at position `i` is divided by `log2(i + 1)`. Position 1 is worth full value, position 10 about a third. Sum these to get **DCG**.
3. **Normalise.** Compute the DCG of the *perfect* ranking - the **IDCG** - and divide. The result sits between 0 and 1.

Step 3 hides a trap that this project cares about a great deal. The ideal ranking must be built from the *complete* list of known-relevant passages, not from the passages the system happened to return. If you normalise by what came back, a system returning one relevant passage and nothing else scores a perfect 1.0 - because that single hit is also the best possible ordering *of that list*. The metric would then reward returning less. There is a regression test in this project named after that failure.

5. The other way: just read everything

Modern language models accept enormous inputs - the one used here takes 1,048,576 tokens, roughly 800,000 words. So why retrieve at all? Why not paste the whole filing into the prompt?

This is the **long-context** approach and it is a genuine alternative, so this project measures it head to head. The trade-offs:

- **Cost.** You pay per token of input. A retrieval prompt is a few thousand

tokens; a whole filing is over a hundred thousand.

- **Latency.** More input takes longer to process.
- **Accuracy.** Models are known to lose track of information buried in the middle of very long inputs.
- **Citations.** If the model got one enormous blob, it cannot tell you *which part* it used. Retrieval knows exactly which passages it supplied.

Part II - What was built

6. Choosing a corpus

The corpus is **120 SEC 10-K filings: 30 companies x 4 consecutive fiscal years** - 68.8 million characters, 14,537 extracted tables.

Four deliberate choices:

Annual reports, not Wikipedia. Filings are dense, table-heavy, full of cross-references ("see Note 12"), and structured by a mandated Item 1-15 hierarchy. Table extraction is where naive pipelines visibly fail.

Four years per company, not one. This is the most important corpus decision. Consecutive annual reports repeat their structure almost verbatim while the figures change. So "R&D expense in fiscal 2025" has three near-identical distractors differing only in the numbers. Retrieval must discriminate on the *year*. A single-year corpus would make the task far too easy and every configuration would score alike.

HTML, not PDF. Filings are *born* as HTML with explicit `<tr>`/`<td>` structure including colspan and rowspan. Converting to PDF and running a layout model throws that away and forces the model to re-infer cell boundaries from pixel positions - reintroducing error into exactly the thing being measured.

A committed ticker list, not a query. "The 30 largest US companies" resolves differently every quarter, so results against it could never be reproduced. The tickers are hardcoded; CIK identifiers are resolved from SEC's official mapping at build time.

7. The central design decision

> **Gold labels anchor to character spans in the document's canonical text, never to chunk identifiers.**

Here is why this is the load-bearing decision in the entire project.

Retrieval does not operate on whole documents - a 500,000-character filing is far too big to hand to anything. Documents are split into **chunks** of a few hundred words each. And *how* to chunk is one of the things being measured, so the chunker changes between configurations.

Now: if a gold label said "chunk 47 of document X is the answer", then switching from fixed-size to structure-aware chunking would renumber every chunk. Chunk 47 would be a completely different piece of text. The evaluation set would silently become wrong, and - this is the dangerous part - **every number would still look plausible.** Nothing would error. You would publish a table comparing systems against different ground truth while believing they shared it.

So labels record character offsets into the document's canonical text, which no chunker is permitted to alter. A chunk is judged relevant if it *covers* the gold span. Consequences:

- The same eval set scores every chunking configuration without modification.
- Adding a new chunker later requires no relabelling.
- The canonical text is immutable. All normalisation happens once, during

parsing, *before* any offset is assigned.

The trap that follows from it

Coverage needs a threshold - a chunk counts as relevant if it contains at least 50% of the gold span. But what if a gold span is *longer* than any chunk a configuration produces?

Then no chunk reaches the threshold, the query has no relevant unit, and it is **silently dropped** from that configuration's average.

That is a measurement disaster wearing the clothes of a reasonable default. Small-chunk configurations would drop every query whose answer is a long table - precisely the queries they fail hardest at - while structure-aware chunking keeps them. The two would be compared on *different query sets*, and small chunks would look better than they are.

Two defences, both implemented: **reachability** is reported per configuration (what fraction of gold passages it can represent at all), and the headline comparison runs on the **intersection** of queries every configuration can score.

8. Module by module

```
corpus/     fetching and parsing filings into canonical text + structure
chunking/   three strategies for splitting documents
evalset/    building, checking, and applying the labelled benchmark
index/      BM25, dense, fusion, reranking
metrics/    nDCG / Recall / MRR, and the statistics for reporting them
llm/        cached, quota-tolerant access to a language model
generation/ answering questions and scoring the answers
ablation/   the experiment grid and its runner
service/    FastAPI API and the inspection UI
```

`corpus/models.py` - the canonical `Document`, its `Blocks` (paragraph, heading, table, boilerplate), and the `Span` algebra. Span coverage is deliberately **asymmetric**: it asks "how much of the gold passage does this chunk contain?", not "how similar are these two spans?". A symmetric measure like Jaccard would penalise a large chunk that fully contains a short answer, which is a *successful* retrieval.

`corpus/edgar.py` - a polite, cached, resumable SEC client. Rate-limited to well under SEC's published ceiling, with a User-Agent naming a real contact (verified necessary: a generic `python-httpx/0.27` User-Agent is refused with HTTP 403). Every fetch is cached, so re-running costs no network traffic.

`corpus/html_parse.py` - the hardest module. Turns filing HTML into canonical text with tables preserved. Discussed at length in Part III, because most of it was arrived at by being wrong first.

`chunking/` - three strategies behind one interface. *Fixed-size* packs words to a token budget with overlap, ignoring structure. *Structure-aware* never splits a table and never crosses a section boundary. *Semantic* embeds each sentence and breaks where consecutive meanings diverge.

`index/bm25.py` - BM25 written by hand rather than taken from a library, for two reasons. The tokeniser has to understand money: a filing writes `34,550` and a question might write `34550`, and the usual lowercase-and-split treats those as unrelated strings on a corpus whose answers are almost all numbers. And the lexical arm is the baseline the headline claim is measured against - a weak or misconfigured BM25 would manufacture the conclusion that hybrid retrieval wins.

`index/dense.py` - exact brute-force cosine search, deliberately *not* approximate. An approximate index has its own recall error, and that error lands inside the number the ablation is trying to read. A configuration could look better or worse because of how the graph was built, and no amount of repetition would separate that from a real retrieval difference. At 42,215 chunks the exact matrix is 152 MB and a query is one matrix-vector product.

`metrics/stats.py` - the module that turned out to matter most. A 15-row table of bare point estimates is not evidence. Two choices:

- A **paired randomisation test**, because both systems see identical queries and pairing removes query difficulty from the variance. Smucker, Allan & Carterette (CIKM 2007) treat it as the reference method for IR.
- **Holm-Bonferroni correction** across the whole family, because comparing 14 configurations against one baseline at alpha=0.05 carries roughly a 51% chance of at least one false positive.

Part III - Decisions that turned out wrong

This is the most useful section. Every item is a real mistake made during construction, how it surfaced, and what replaced it.

9. Parsing: five wrong turns

9.1 Feeding the parser a decoded string

The first parser took `str`. The very first real filing failed: inline-XBRL filings are XHTML carrying their own `<?xml encoding=...?>` declaration, and lxml refuses a `str` that declares an encoding - rightly, because the string has already been decoded by somebody's guess.

Worse than the crash was what would have happened without it: decoding first overrides the filing's own declaration and mojibakes the typographic characters filings are full of.

Fix: bytes travel from socket to cache to parser, so exactly one component honours the declaration. Undeclared bytes get an explicit UTF-8 parser, because lxml's fallback is a legacy single-byte encoding that turned "Café" into "CafÃ(c)".

9.2 Assuming header rows contain no digits

Header detection assumed a header row has no numbers in it. Reasonable-sounding, and wrong on exactly the tables that matter: **the headers of a financial statement are years.** Every income statement was classified as having zero header rows, stripping the column labels that the row-sentence rendering depends on and leaving bare numbers with nothing to match a query against.

Fix: a header row is one whose *stub cell is empty*. Filings put the row label in column zero and leave it blank while column headers are declared. Close to universal, and it survives multi-level headers.

9.3 Rendering tables straight from the parsed grid

Filings *lay tables out* rather than tabulating them. A three-column financial table arrives as thirty physical columns: currency symbols, percent signs, and sub-1%-width spacer columns each occupy their own cell. Rendered naively this produced rows like `| R&D | $ | 34,550 | | | | 10 | % |` and header rows with the same value repeated fifteen times.

Fix: drop columns empty across all *data* rows - judging emptiness on all rows keeps every spacer alive, because a spanning header is repeated across the spacers it covers - then merge colspan groups back into single cells.

9.4 Collapsing repeated cells by matching text

The obvious way to undo colspan smearing is to collapse adjacent cells with equal text. It fixes a row label spread across three columns - and **silently deletes one of two fiscal-year columns that happen to hold the same figure.**

Fix: record which source `<td>` produced each physical column, and collapse on shared *origin* rather than shared text. Same origin means one cell was stretched; different origins mean two cells coincidentally agree. Exact rather than heuristic.

A related bug: the fold initially skipped empty cells, which left header rows three columns wider than their data rows, so every column label lined up against the wrong value.

9.5 Suppressing the table of contents one heading at a time

A 10-K lists every heading in a contents block, then repeats them in the body. Naive heading detection finds each twice.

First attempt: suppress any heading followed by little text. This deleted `PART I` (always immediately followed by `Item 1`, so the gap is tiny by construction), one-line sections like `Item 4. Mine Safety Disclosures`, and short-but-real ones like `Item 2. Properties`. Losing the Part headings meant *no passage in the corpus carried a Part at all.*

Second attempt: require a *run* of tightly packed headings. Better, but the run does not stop at the end of the contents block - it continues into the body and swallows the first real `PART I` and `Item 1`.

Fix: require both a tight run *and* that the heading's text appears again later. That is what a table of contents *is*. The final occurrence - the body heading - is always kept.

10. The corpus was 15% garbage

Every chunker reported a maximum chunk of **131,551 tokens**. For a 512-token target that is only possible if a single whitespace-free "word" is half a megabyte long. It was.

Inline-XBRL filings carry a machine-readable payload inside a `display:none` wrapper. In one filing it was **526,296 characters - 31.7% of that document** - entirely taxonomy URIs, context identifiers and period dates. Across the corpus, **12.2 million characters, 15% of everything ingested.**

The fix targets `ix:header`. What matters is what is deliberately *not* removed: `ix:nonfraction` and `ix:nonnumeric` **wrap the visible content** - one filing had 11,042 of them, one around every reported figure. Dropping inline XBRL wholesale would have deleted every number in the corpus while leaving the prose intact.

Longest "word" after the fix: **62 characters**, down from 526,205.

11. Two corpus gaps that failed silently

Neither raised an exception. Both were found by counting documents per company.

Pagination. SEC's `filings.recent` holds only a company's most recent 1,000 filings. JPMorgan's window contained 25,601 filings covering barely one calendar year and exactly *one* annual report, with 68 further pages unread. The four banks contributed one document each instead of four.

Successor entities. The ticker-to-CIK map returns the *current* registrant. After a reincorporation that is a successor with no filing history. `XOM` resolves to a CIK holding no annual reports at all; every ExxonMobil 10-K sits under CIK

34088. BlackRock's history is split across two CIKs.

Overrides are listed explicitly rather than inferred from former names - guessing a predecessor CIK is exactly how a corpus ends up quietly holding a *different company's* filings, which is far worse than a recorded gap.

12. An experimental axis that was silently inert

The table-rendering configuration reported nDCG@10 **identical to its markdown twin to four decimal places, with the same chunk count.**

Rendering is applied during *parsing*, and the runner only threaded it through the index key - so the row-sentence configuration silently reused the markdown corpus. A plausible number for an experiment that never ran. This is precisely the class of failure the project is built to catch, and it took reading the output rather than trusting it.

The fix exposed a genuine collision: rendering changes the canonical text, so it changes every character offset, so gold spans built against one rendering do not apply to another. Resolved by re-parsing from cached bytes and re-deriving gold spans - which works only because query IDs are content-addressed on (document, row label, period) rather than on offsets.

13. The lenient-labels fix that was backwards

The IR literature on **pooling bias** warns that nDCG treats unjudged retrieved documents as non-relevant. This eval set labels exactly one gold row per query, while a filing states the same figure in the income statement, the MD&A prose, and often a segment table.

Measured: **15.3%** of queries have an unjudged top-10 chunk carrying the answer; **11.6%** are scored as complete misses *despite the answer being retrieved*.

So a "lenient" judgement set was built - any chunk of the gold document containing the figure counts - expecting it to bound the true score from *above*.

It does the opposite. Lenient nDCG@10 fell from 0.1912 to 0.1830; Recall@50 from 0.5324 to 0.3623.

That is arithmetic, not a retrieval result, and it traces directly back to §4: IDCG is computed from the *complete* judgement set. Widening the judgements adds relevant documents the retriever mostly did not return, so the ideal ranking improves while the actual one barely moves. Recall falls because its denominator grew. Worked through, with the gold at rank 3 of 5 and four duplicates present but not retrieved:

```
strict    nDCG@10 = 0.5000   Recall = 1.0000   MRR = 0.3333
lenient   nDCG@10 = 0.3031   Recall = 0.2000   MRR = 0.3333
```

Only MRR is a valid signal here, because it depends solely on the rank of the first relevant document and cannot be diluted by relevant documents never returned. Under lenient judgements MRR rises 0.1745 -> 0.2396, and *that* is the evidence the strict labels under-credit retrieval.

Publishing lenient nDCG as an upper bound would have been a real methodological error, reached by reasoning that sounded correct. Four assertions now pin the arithmetic so nobody "fixes" it back.

14. Infrastructure decisions that were wrong

Dependency floors set to whatever the dev machine had. `numpy>=2.1` would have forced pip to upgrade

numpy inside a Kaggle session, breaking the ABI of the preinstalled torch - surfacing much later as an unrelated import error, after half an hour of corpus rebuilding. The code only needs numpy 1.17-era APIs.

Four unused runtime dependencies. `qdrant-client`, `fastapi`, `uvicorn`, `jinja2` were declared required and imported nowhere, dragging grpcio, protobuf and uvloop into every environment.

A wrapper that broke on a version pairing. `sentence-transformers` 5.x hands the tokenizer's whole `BatchEncoding` to the model as positional `input_ids`; `XLMRobertaModel.forward` then evaluates `input_ids.device`, a dict lookup that misses, and raises a **bare `AttributeError` with no message** twelve frames down. It failed on a T4 after 19 minutes of successful embedding. Fixed by calling transformers directly - BAAI's own documented usage - removing the layer rather than pinning around it.

Positional alignment of precomputed vectors. One filing parsed 360 characters longer on the GPU worker than locally. Because chunk IDs encode character spans, one chunk got a different ID. Positional alignment matched 42,214 of 42,215 and would have shifted every subsequent row by a document boundary - assigning one company's vectors to another's chunks, silently. Fixed by aligning on ID.

And the explanation for that drift was wrong. It was recorded here, in the README, and in a commit message as "SEC re-posted the filing" - a plausible story, since EDGAR does re-post filings, and one that was never checked. Re-fetching both disagreeing documents showed their raw bytes byte-identical to the committed manifest, last modified in 2023. Nothing upstream had changed. **The parser was not deterministic across machines**, and two independent causes were producing it:

- Microsoft's filing writes `&#149;` five times as a list bullet. HTML5 says

numeric references in `0x80-0x9F` are reinterpreted through Windows-1252, so `&#149;` means `-`. Whether libxml2 does this depends on its version; older builds hand back `U+0095`, a meaningless C1 control character. Five characters out of 357,277 - the document length never changed, so only the digest moved.

- Southern's filing is 19.6 MB, and libxml2 enforces an internal size ceiling.

When it trips it does not raise: it stops adding nodes and returns a tree that looks complete. The filing lost its closing paragraph and eleven elements, and the ceiling differs between libxml2 releases - hence two machines, two corpora.

Both are now handled explicitly rather than inherited from whatever libxml2 is installed: the C1 range is mapped in `normalize_text`, chosen because it is the one place all text passes through *before* any offset is assigned, and every parse sets `huge_tree`. After rebuilding, the local corpus reproduces the GPU worker's output exactly and the vector files align with zero orphans.

The lesson is not about libxml2. A checksum told me two documents disagreed, and I explained the disagreement instead of investigating it. The explanation was consistent with every fact I had, cost nothing to believe, and sent me looking in the wrong place - at EDGAR, which was blameless - while a reproducibility bug sat in the parser. Re-fetching the bytes took two minutes and falsified it outright. A cheap test that could have contradicted the story was available the whole time, and the story's plausibility is exactly why it did not occur to me to run it.

There is a second-order trap here too. The first rebuild after the fix reported *zero* documents changed, which looked like a refutation of the diagnosis. It was not: `ingest()` caches parsed documents in `data/interim/` and skips re-parsing, so a parser fix does nothing until that cache is cleared. A fix that appears to have no effect is not evidence the diagnosis was wrong until you have checked that the fix actually ran.

A self-referential integrity check. That drift exposed a hole in a safeguard this project *advertised*: `ingest()` writes the manifest and `load_corpus()` verifies against whatever manifest is on disk. A fresh ingest followed by `load_corpus` verified a run against its own output. The GPU worker printed "corpus verified: 120 documents" while holding a document that differed from the committed one.

And the fix for it was half a fix - which is the more useful lesson. The worker was changed to read the committed manifest into a variable *before* calling `ingest()`, and that was recorded as done. It was not: the snapshot was read, printed as a document count, and never compared to anything. The self-referential check downstream was untouched and still always passed. This survived until the vector files were audited chunk id by chunk id against a fresh local rebuild, which is when the one stale id turned up - the drift had been shipped, unnoticed, in an artifact the repository presents as verified.

Two things are worth taking from it. First, a safeguard that cannot fail is indistinguishable from no safeguard, and reading the right value is not the same as checking it; the code *looked* like a verification because a variable named `committed` appeared next to a print statement. Second, the thing that actually contained the damage was not a check at all but a representation choice - keying vectors by chunk id meant the mismatch had somewhere to show up as a count. Defences that make a whole class of error *structurally visible* outrank defences that test for it, because the test is only as good as the last person's attention and the structure holds regardless.

Query vectors reused across a rewrite of the query. Paraphrasing keeps `query_id` deliberately - that is what makes the two eval sets comparable. The artifact loader keyed vectors by id and then filed each row under whatever text the caller currently held, so the paraphrased run scored every dense arm with vectors embedded from the *original* wording. No exception, complete coverage, entirely ordinary-looking metrics. The only symptom was nDCG@10 matching the original run at four decimal places, which is the sole reason it was caught.

The artifacts now record the text each vector was built from, rows whose text has changed are dropped, and an artifact that cannot say what it embedded is refused outright. An unmeasured arm costs a re-run; an arm silently scored against stale vectors costs the credibility of every number printed beside it.

Worth naming the pattern, because this project produced it five separate times: a mechanism that appears to validate something and does not. A manifest check that verified a run against its own output. A snapshot read but never compared. A reuse shortcut that skipped the write it was guarding. A `.gitignore` rule naming one directory when the next download used another. And this. None raised an error; all five reported success. The common shape is that the check and the thing being checked were allowed to come from the same source, so agreement was guaranteed rather than earned.

An undefined metric rendered as a real zero. The long-context arm showed citation precision `0.000` beside retrieval's `0.567`, reading as "long context cites badly". It *cannot* cite a gold chunk - its context is one whole-document pseudo-chunk. An empty gold list made precision compute as a genuine `0.0`. Now `None`, printed as "not measured".

Part IV - What was measured

15. The retrieval ablation

Full corpus, 42,215 chunks, 143 queries judgeable by every configuration.

configuration	nDCG@10	95% CI	Recall@50	delta vs base	p (Holm)
rerank-candidates-50	0.2145	[0.160, 0.273]	0.5245	+0.0192	1.000
rerank-candidates-25	0.2103	[0.157, 0.268]	0.5245	+0.0150	1.000
rerank-bm25-100	0.2057	[0.153, 0.263]	0.5385	+0.0104	1.000
hybrid-plus-rerank	0.2003	[0.148, 0.256]	0.5455	+0.0050	1.000
retrieval-hybrid-rrf	0.1991	[0.146, 0.254]	0.4965	+0.0039	1.000
baseline-bm25-fixed512	0.1953	[0.143, 0.251]	0.5070	-	-
chunk-struct512	0.1874	[0.134, 0.245]	0.5245	-0.0078	1.000
rerank-candidates-200	0.1854	[0.134, 0.240]	0.5035	-0.0099	1.000
chunk-fixed256o32	0.1699	[0.119, 0.226]	0.4126	-0.0254	1.000
tables-row-sentences	0.1688	[0.122, 0.222]	0.4935	-0.0265	1.000
retrieval-dense-bge	0.1196	[0.077, 0.167]	0.3077	-0.0757	0.130
embed-e5-base	0.0413	[0.019, 0.068]	0.2168	-0.1540	0.001

Finding 0 - the benchmark was hiding the effect it existed to measure

This is the most important result in the project, and it was invisible until the last thing on the list got done.

Every query in this benchmark was generated from a table row and reused that row's label word for word. Paraphrasing rewrites the questions the way a person would ask them, touching nothing else - same corpus, same gold spans, same query ids, same 143 shared queries. Then the grid runs again.

configuration	original	paraphrased	delta vs base	p (Holm)
rerank-bm25-100	0.2057	0.1377	+0.0902	0.0016 ?
rerank-candidates-200	0.1854	0.1342	+0.0867	0.0028 ?
rerank-candidates-50	0.2145	0.1058	+0.0584	0.0036 ?
rerank-candidates-25	0.2103	0.0919	+0.0444	0.0350 ?
baseline-bm25-fixed512	0.1953	0.0475	-	-

On the original queries, nothing favouring any configuration was significant. On the paraphrased queries, **every reranking configuration is significant**.

No retriever changed. The benchmark was giving BM25 an exact string match on 73% of its queries, and a first stage that already has the answer at rank 1 gives a reranker nothing to improve. Removing that advantage did not make reranking better; it made it measurable.

Notice also that the candidate-depth finding **inverts**. Depth 200 was the worst configuration on the original queries and is second best here - consistent with a reranker that is now doing real work, for which a deeper shortlist is an opportunity rather than more chances to err.

The lesson generalises past retrieval. Every piece of statistical machinery in this project was correct: the paired test, the bootstrap intervals, the Holm correction. All of it was applied conscientiously to numbers produced by a benchmark that favoured one arm, and none of it could have noticed, because none of it was wrong. Significance testing protects against reading noise as signal. It offers no protection whatsoever against a well-measured answer to the wrong question. The only thing that caught this was recording lexical overlap per query - deciding, before any of the results existed, to measure the confound rather than argue about it.

Finding 1 - nothing was shown to help; one thing was shown to hurt

Reranking takes the top four slots, and **no improvement survives Holm correction** - the best configuration's corrected p is 1.000. At 143 queries the CI spans about +/-0.055, so a 0.019 gap is inside the noise.

Writing "reranking lifted nDCG@10 from 0.195 to 0.215" would be correct arithmetic and a false finding.

Exactly one comparison in the grid does survive: `embed-e5-base` at 0.1540 *below* baseline, p = 0.001. That asymmetry is worth sitting with. The study was built to detect improvements and detected none; the only thing it could resolve at this sample size was a configuration failing badly. Significance is a statement about effect size relative to noise, not about importance, and a benchmark too small to confirm the effect you are hoping for is still perfectly capable of confirming one you are not.

A result that large invites a bug hypothesis, and E5 has an obvious one: it requires literal `query:` and `passage:` prefixes and loses a lot of accuracy without them. The prefixes were verified present in the code path, and then the claim was checked rather than trusted, by asking where the gold passages actually land. Random ranking among 42,215 chunks would put gold in the top 1,000 for about 2.4% of queries; E5 manages 59.3%, against BGE-M3's 71.8%. E5 is working. It is just worse here - and the next section explains most of what "here" is doing.

Finding 2 - the average hides a large real effect

Splitting queries at 0.4 content-word overlap with their gold passage:

configuration	low-overlap	vs base	high-overlap	vs base
baseline	0.1091	-	0.2254	-
rerank-bm25-100	0.2309	+111.7%	0.1969	-12.6%
rerank-candidates-200	0.2104	+92.9%	0.1767	-21.6%
rerank-candidates-50	0.1937	+77.6%	0.2218	-1.6%
retrieval-hybrid-rrf	0.1374	+25.9%	0.2207	-2.1%
retrieval-dense-bge	0.1346	+23.4%	0.1143	-49.3%
embed-e5-base	0.0467	-57.2%	0.0394	-82.5%

The cross-encoder roughly doubles performance where the question does not share wording with its answer, and slightly hurts where it does. Exactly the right shape: where BM25 already has an exact string match there is nothing to fix, and reordering can only push a correct top hit down. The +0.019 average is a large positive on half the queries cancelled by a small negative on the other half.

The dense rows make the same point from the other direction, and they are the reason the overall dense numbers should not be read as "embeddings are bad at finance". `retrieval-dense-bge` is the **only** configuration in the whole grid whose low-overlap score *exceeds* its high-overlap score - 0.1346 against 0.1143. Every lexical configuration roughly doubles in the other direction. That is the signature of a method that does not match on strings: it gains nothing when the query reuses the document's wording, and loses nothing when it does not.

Which means the benchmark is not neutral between the two families. Its queries are generated from table rows and reuse the row labels verbatim, handing BM25 an exact match on most of them. On the low-overlap subset - the queries that look more like something a person would type - `retrieval-dense-bge` (0.1346) beats the baseline (0.1091), and `retrieval-hybrid-rrf` (0.1374) beats everything except reranking. The defensible claim is "BM25 wins on queries phrased in the filing's own words", not "BM25 wins on SEC filings", and the difference between those two sentences is the entire value of having measured the split.

This is also the clearest argument for query paraphrasing, which is not done. A benchmark that systematically advantages one arm will report that arm winning, and will do so with tight confidence intervals and a straight face.

Finding 3 - deeper shortlists are worse

depth	nDCG@10	recall ceiling
25	0.2103	44.4%
50	0.2145	53.2%
100	0.2057	61.6%
200	0.1854	73.6%

Depth 200 has by far the **best** ceiling and the **worst** score - below no reranking at all. The cross-encoder is not failing to *see* the answer; given more candidates it actively promotes wrong ones past it. The intuitive tuning move - widen the shortlist - measurably backfires.

16. Label quality, and whether the conclusions survive it

All 216 labels were audited by a language model asked to reject on specific checkable grounds. **44 of 216 rejected - 20.4%**, 0 unparseable. Rejections concentrate on entity/place names used as subjects ("united states", "duke energy ohio") and on passages with several figures under one label.

Re-running the whole grid on the 172 accepted labels:

- Twelve of thirteen configurations gain **+0.020 to +0.031** - the signature of labels that were genuinely unanswerable, penalising all systems about equally. The exception is `embed-e5-base`, which *loses* 0.0035: bad labels were not what was holding it back.
- **The ranking is stable, not identical.** Top three and bottom four unchanged; `retrieval-hybrid-rrf` and `hybrid-plus-rerank` trade ranks 4 and 5 across a gap of 0.0025. That is noise, and is why neither is reported as a finding.
- **The significance picture is unchanged**: no improvement survives Holm on either label set, and `embed-e5-base` is the single significant comparison at p = 0.001 on both.
- The overlap effect gets *stronger*: `rerank-bm25-100` goes from +112% to **+171%** on low-overlap queries, and `retrieval-dense-bge` inverts harder still - 0.2212 low against 0.1220 high, now **beating the baseline's low-overlap 0.1276 by +73%** while trailing it badly overall.

These are marked `MODEL_CHECKED`, never `HUMAN_VERIFIED`, and a test enforces it. A model auditing labels a program generated from the same tables is not an independent second opinion, and cannot see that a *different* passage would have been the better gold.

17. Retrieval versus long context

`gemini-3.6-flash`, 12 of 216 queries, costs computed from the API's own reported token counts.

	retrieval (top-10)	long context (whole filing)	ratio
mean prompt tokens	7,345	130,701	17.8x
cost per query	$0.011224	$0.196322	17.5x
p95 latency	4.54 s	8.54 s	1.9x
accuracy, all queries	0.300	0.556	-
refusal rate	5 of 10	0 of 9	-

The brief's "roughly 1,250x cheaper" is not reproducible - measured 17.5x. 1,250x requires assuming a 1M-token context, an ~800-token retrieval prompt, *and* zero output cost.

Long context currently wins on accuracy, and the refusal column says why: retrieval declined half the questions because the answer was not in its top-10. When it did answer it was slightly better (0.600 vs 0.556) and it cited sources, which the long-context arm structurally cannot. Retrieval is cheaper and faster and loses on accuracy *because its first stage is weak*.

Two caveats that cut against the result: long context is **handed the correct filing** while retrieval must find it among 120, and 12 queries is a small sample.

18. Operating findings

GPU throughput, Tesla T4, 42,215 chunks each: BGE-M3 13.9 min (51 chunks/sec); multilingual-E5-base 4.8 min (149/sec).

Gemini free tier, measured not assumed: `batchEmbedContents` caps at **32** texts (64 -> `RESOURCE_EXHAUSTED`), roughly 3 requests/min/model - about **96 texts/minute**, or 7.3 hours for one embedding pass. Against Kaggle's ~3,060/min that is **32x**, which settled where GPU work belongs.

Thinking models consume the output budget. `gemini-3.6-flash` at `maxOutputTokens=16` returns **empty text** - 13 tokens went to `thoughtsTokenCount`. At 512 the same prompt answers correctly after 81 thought tokens. `thinkingBudget: 0` is rejected with HTTP 400.

Service: index builds in 31.7 s over 42,215 chunks; `/search` returns in **1.9 ms**.

Part V - Running and extending

19. Running it

```
uv venv && uv pip install -e ".[dev,service]"
uv run python -m ruff check . && uv run python -m pytest
```

422 tests pass offline with no API key and no model download.

Rebuild everything:

```
uv run python -m retrieval_ablation.corpus.ingest      # ~40 min, 120 filings
uv run python -m retrieval_ablation.evalset.build      # deterministic
uv run python -m retrieval_ablation.ablation.runner    # the ablation
```

GPU arms run on Kaggle (`notebooks/kaggle_gpu_arms.py`), because PyTorch cannot load on the development machine under an enforced Windows code-integrity policy - `WinError 4551`, and the log names `torch_cpu.dll`, so the CPU build is blocked too. Disabling that control is machine-wide and cannot be undone without reinstalling Windows.

20. Extending it

A new chunker: subclass `Chunker`, return chunks whose spans index the canonical text. No relabelling is needed - that is the payoff of §7.

A new retriever: implement `Retriever.search`. Return an empty list rather than padding with zero-scoring results.

A new metric: add to `metrics/retrieval.py`. Return `None` when it cannot be computed; never a plausible default.

A new ablation row: add a `Config` to `build_grid()`. A check asserts every non-interaction row differs from its reference on exactly one axis.

21. Honest limitations

1. **Queries are templated**, reusing the corpus's own wording. Median overlap 0.46. Recorded per query and reported split, but paraphrasing would be better.
2. **Labels are model-checked, not human-verified.**
3. **Single gold passage per query** understates retrieval by ~12% (§13).
4. **Six of fifteen configurations unmeasured** - dense, hybrid, embedding models, semantic chunking. The GPU run produced passage vectors but not query vectors, and both sides must come from the same model.
5. **Generation measured on 12 of 216 queries**, quota-bound. Cost ratios are solid; accuracy figures are indicative.

6. **Faithfulness not measured at all.**
7. **Approximate token counting** (characters ÷ 4) rather than a real tokenizer.

Every configuration uses the same counter, so comparisons are valid, but boundaries differ from a model's own.

8. **One-axis-at-a-time design cannot detect interactions.** One crossed cell is run explicitly.
9. **Absolute numbers are not comparable to published benchmarks** - this corpus is deliberately adversarial.

Glossary

BM25 - lexical ranking function combining rare-word weighting, term-frequency saturation, and length normalisation.

Bi-encoder - model embedding query and passage separately, compared at the end. Fast, precomputable, cannot model interaction.

Chunk - a retrievable unit, a few hundred words, carved from a document.

CIK - SEC's numeric identifier for a filing entity.

Cross-encoder - model reading query and passage jointly and scoring the pair. Accurate, expensive, usable only over a shortlist.

DCG / IDCG / nDCG - Discounted Cumulative Gain; the Ideal DCG of a perfect ranking; their ratio. See §4 for the IDCG trap.

Embedding - a list of numbers representing meaning, such that similar texts get similar lists.

Gold passage - the passage labelled as containing a query's answer.

Holm-Bonferroni - a correction controlling the family-wise error rate across many comparisons; uniformly more powerful than plain Bonferroni.

Inline XBRL - machine-readable financial tags embedded in filing HTML.

MRR - Mean Reciprocal Rank; the average of 1/(rank of first relevant hit).

Pooling bias - the distortion from metrics treating unjudged documents as non-relevant.

qrels - query relevance judgements: which passages are relevant to which query, at what grade.

Reachability - the fraction of gold passages a chunking configuration can represent at all.

Recall ceiling - the first stage's recall at the reranking shortlist depth; the hard upper bound on what reranking can achieve.

RRF - Reciprocal Rank Fusion; combines rankings by position, avoiding incomparable scores.

Span - a half-open character interval `[start, end)` into a document's canonical text.

10-K - a US company's annual report to the SEC.